

COS30017 - Software Development for Mobile Devices

Assignment 2: Car Rental App

Nguyễn Chí Cường - 104852250



Git repository: <https://github.com/Opatzo2212/COS30017.git>

1. My development plan

For this assignment, I first did research about similar car rental sites. I wanted to get a feel of how their website worked and how the car rental flow is. From my research I have found a website that fits my development idea. Here's how the website looked like:

[Home](#) / [All Rental Vehicles – United States](#) /
[All Vans – United States](#) /
7 Passenger Minivan Rental – United States

7 Passenger Minivan Rental – United States

A minivan rental is perfect for long [road trips](#) and family vacations with room for 7 people. Reserve now and get low

Show More 



Chrysler Pacifica or similar

 Automatic  7 People  5 Bags

Vehicle Features

- AM/FM Stereo Radio
- Cruise Control
- 2 Wheel Drive
- Gasoline Vehicle
- Air Conditioning



Though the website is made for desktop, it gave me inspiration to recreate something similar in Android Studio.

My first focus was to work on the app UI. I first created a temporary UI where it displays everything in a single xml file. However I soon decided to split the website into 3 separate

parts. The main activity includes the UI for every other feature beside the cars and their information:



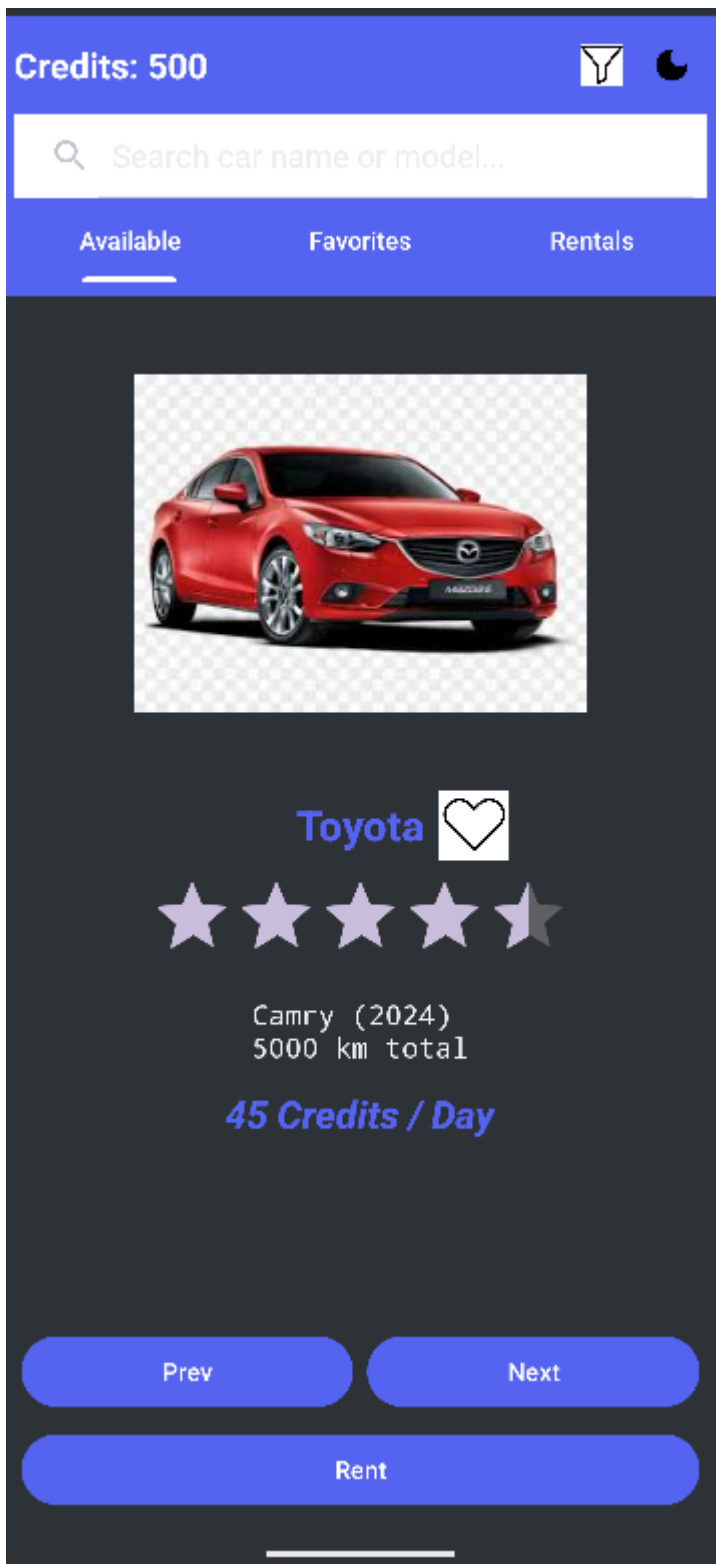
The second file is a fragment that displays different types of cars depending on the current index. The third is a fragment for when renting the car.

After the UI was done I moved on to the features implementation. The first feature was the car rental feature. The main features to implement were the Parcelable objects that allow the passing of data. In my application, the Car data is passed between the MainActivity and RentalActivity as a Parcelable object. While Java offers the standard Serializable interface, Parcelable is the Android-optimized implementation for inter-component communication.

The main advantages of using Parcelable objects on android include:

- **Superior Performance:** Unlike Serializable, which uses reflection to identify object fields and generates a large amount of temporary objects during the process, Parcelable is designed to be high-performance by requiring the developer to explicitly define the writing and reading of each field.
- **Memory Efficiency:** Because Parcelable avoids the overhead of reflection and extensive metadata storage, it is significantly faster and consumes less memory, which is critical for mobile devices with limited resources.
- **Type Safety:** The implementation requires a CREATOR field that ensures the object is reconstructed exactly as it was written, providing a robust way to maintain data integrity when moving objects across Intents.
- **Platform Optimization:** Parcelable is a part of the Android Open Source Project (AOSP) specifically designed for the Binder (Android's inter-process communication system), making it the "native" choice for the platform

Afterward I implemented other smaller features like dark mode and search function. The dark mode uses a different style compared to normal mode:



2. User stories and Use cases

User Stories:

Here's some user stories from my app:

- **Story 1: A Frequent Traveler**

- "As a frequent traveler, I want to quickly rent a car to get around whenever I travel to a new destination. With the app, I can easily find my favorite car and rent them to travel around."
- **Story 2: Budget-Conscious Renter**
 - "As a student with a limited credit balance, I want to sort cars by cost (low to high) to find cheap cars whenever I need to get around and impress someone"

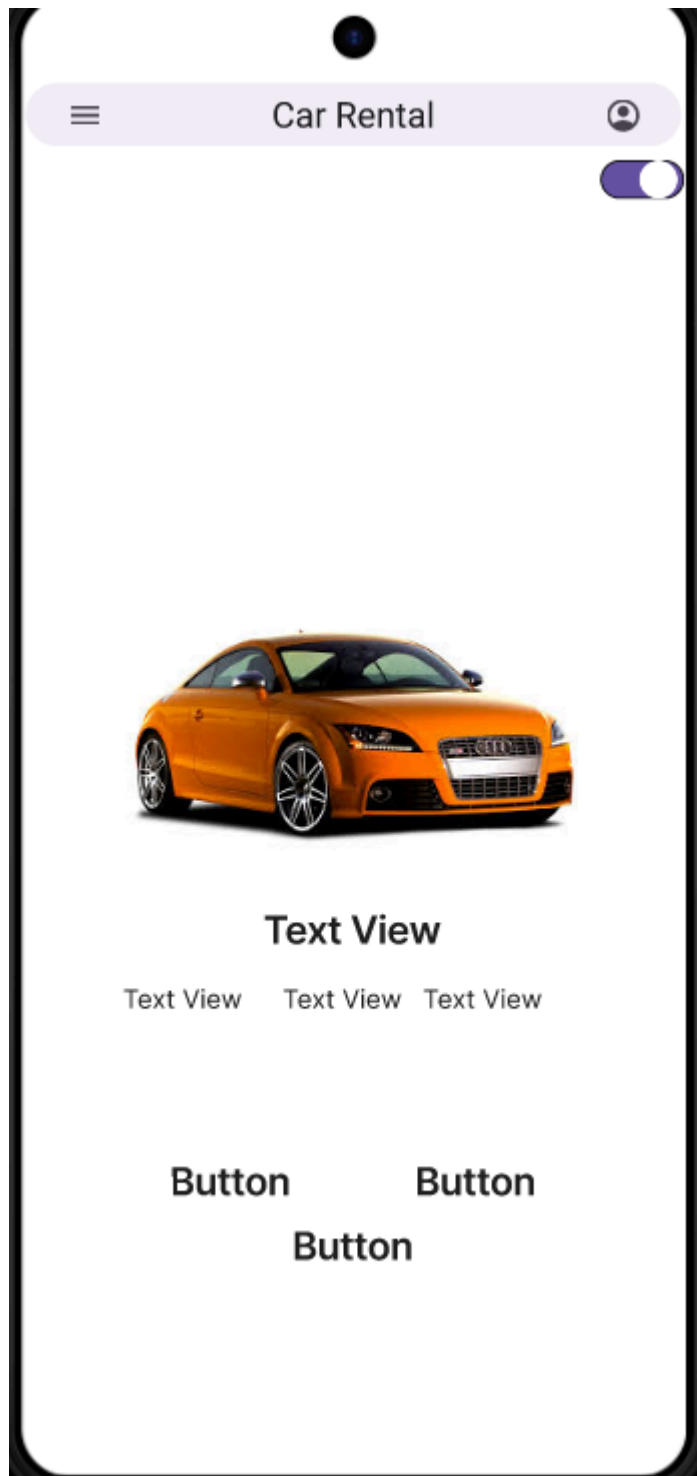
Use Case 1: Renting a Vehicle

- **User:** Customer.
- **Precondition:** User has a credit balance of 500.
- **Main Flow:**
 1. The user browses or searches for a car in the main activity.
 2. The user clicks the '**Rent**' button.
 3. The user selects the number of days via a **Slider** on the Rental screen.
 4. The user clicks '**Save**' to confirm the rental.
- **Postcondition:** The car is removed from the available list, and the user's credit balance is updated.

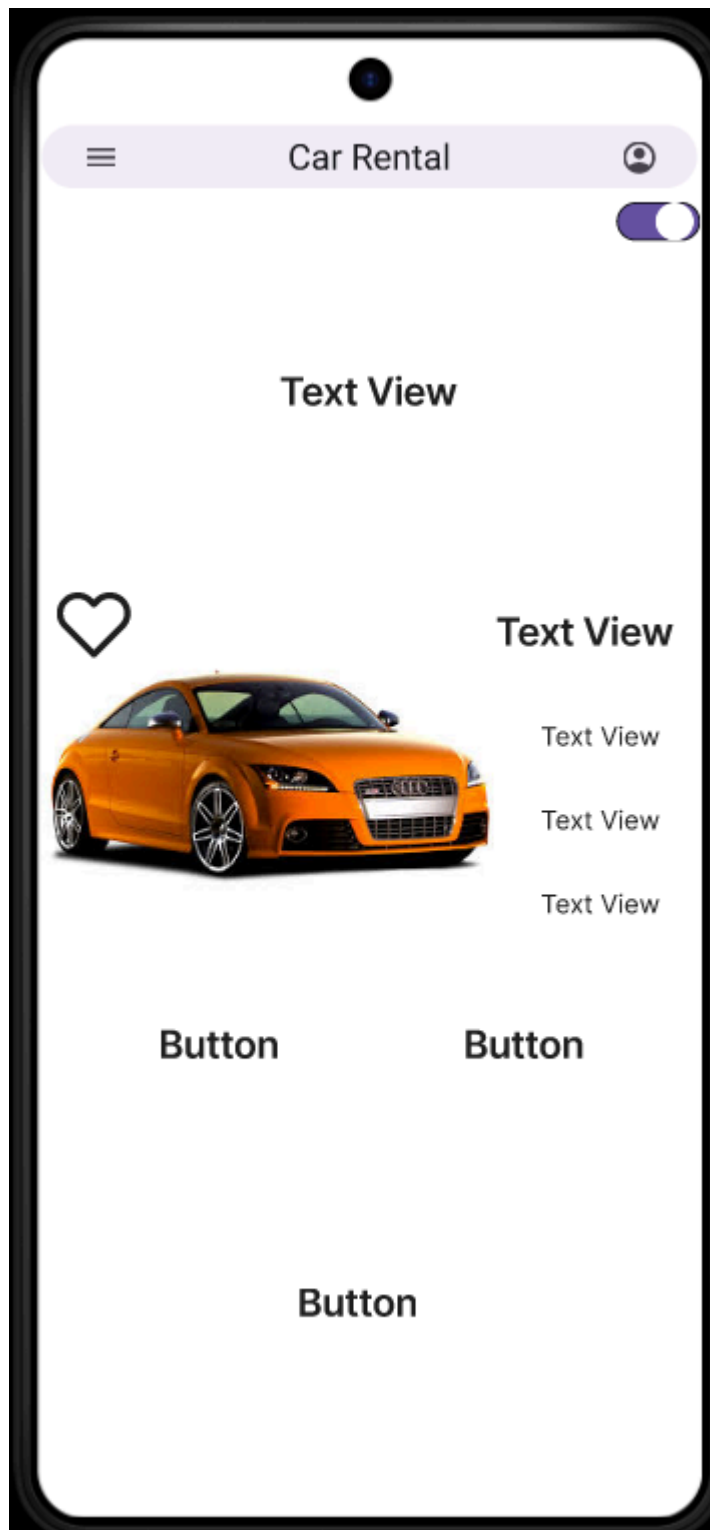
Use Case 2: Managing Favorites

- **Actor:** Client.
- **Precondition:** User is viewing the available car list.
- **Main Flow:**
 1. The user identifies a car they like and clicks the **Heart Icon**.
 2. The user selects the '**Favorites**' tab at the top of the screen to view their saved list.
 3. If the car is later rented, it appears in this list with a "**Currently Rented**" overlay.
- **Postcondition:** The vehicle is tagged as a favorite in memory for quick access.

Possible layout sketches include:



This UI is good if the user wants to look at how the Car looks like first before looking at its information.



This UI is good if the user wants to know all the information when browsing.

3. Key decisions made

- **Parcelable Objects:** I implemented the Parcelable interface for the Car data class. This is the Android-preferred method for passing complex data between activities via

Intents, as it offers significantly better performance by reducing overhead during serialization.

- **Tabbed Fragment Navigation:** I chose a TabLayout with a FragmentContainerView to manage the three distinct views: Available, Favorites, and Rentals. This decision keeps the UI organized and allows the search and sort functions to update the displayed list dynamically without full activity restarts.
- **Non-TextView Widgets:** To enhance interactivity, I utilized a RatingBar for car reviews and a Slider (restricted to 1–7 days) for the rental duration selection, meeting the requirement for non-standard UI components.

4. Issues encountered

Issue Encountered	Solution
Sorting Integrity: Sorting the list sometimes affected the base dataset, making it difficult to return to the default view.	Used Kotlin's <code>.sortedBy</code> and <code>.filter</code> functions to create a new temporary list for the display layer without modifying the original master list.
Rental Limit Logic: Ensuring a single rental did not exceed 400 credits while the total balance stayed under 500.	Added a validation check in RentalActivity that prevents the user from clicking 'Save' if the total (Daily Cost × Days) exceeds the limit.

I also have encountered smaller issues when implementing my fragment logic and other small problems. Though they weren't noticeable enough to be noted down.

5. Reflection

What Worked Well

- **Incremental Feature Building:** Building the core car display logic before adding search, sort, and favorites allowed me to ensure the navigation was stable before introducing data filters.
- **Task Management:** By breaking down the assignments problems into many smaller parts, it was much more manageable.

What Could Be Improved

- **MVVM Architecture:** While functional, using a ViewModel would have been a more modern approach to handling state persistence than relying on activity configuration changes.
- **UI Feedback:** I would like to implement more robust feedback, such as a custom Snackbar with an "Undo" action when a car is rented or favorited, to give the user better immediate control.
- **Better Time Management:** The time management in this assignment wasn't the greatest and I wasn't able to implement some features I would have liked to.